ENGINEERING TECHNICAL MANUAL & SPECIFICATION

Sign Connect AI

Comprehensive System Architecture, Deep Learning Implementation, WebRTC Video Streaming, NLP Grammar Engine & Complete Setup Guide

PROJECT NAME	**DOCUMENT VERSION**
Sign Connect AI (SignConnect)	v2.4.0 (Production Release)
CORE TECHNOLOGIES	**MODEL DATASETS**
PyTorch, MediaPipe, WebRTC, Flask-SocketIO, Python 3	SIBI / Indonesian-Indian Sign Language (A-Z, 1-9, Gestures)
ARCHITECTURE PATTERN	**DEPLOYMENT TARGETS**
Decoupled Producer-Consumer AI Worker & P2P Mesh	Localhost, Cloudflare Tunnel Edge, Multi-Device Web

Sign Connect AI Project Team • Confidential & Proprietary

Date: October 2026

Table of Contents

01 Executive Summary & Project Motivation Page 3

02 High-Level System Architecture & Component Design Page 4

03 Computer Vision & Deep Learning Subsystem Page 6

04 Real-Time WebRTC Video & Socket.IO Signaling Page 10

05 Natural Language Processing & Autocomplete Pipeline Page 12

06 Interactive Avatar, Stickers & Audio Services Page 14

07 Complete Codebase & File-by-File Breakdown Page 16

08 Step-by-Step Installation & Execution Guide Page 19

09 Complete Socket.IO Protocol & API Reference Page 22

10 Automated Testing, Regression Suites & Troubleshooting Page 24

1. Executive Summary & Project Motivation

Sign Connect AI is an advanced, real-time, bidirectional assistive telecommunication platform engineered to bridge the communication barrier between deaf or hard-of-hearing sign language users and non-signing individuals during live video calls. While standard video conferencing tools (Zoom, Microsoft Teams, Google Meet) provide passive video streams, they lack native comprehension of sign language gestures. Sign Connect AI integrates computer vision, deep convolutional neural networks (CNNs), natural language processing (NLP), speech synthesis, and WebRTC streaming directly into a zero-install browser environment.

> **Core Value Proposition**
>
> A single, seamless interface where a user can make signs in front of their webcam, have those signs classified at 20+ FPS with sub-50ms latency, buffered into words, grammatically smoothed into fluent sentences, spoken aloud via Text-to-Speech (TTS), and streamed over peer-to-peer WebRTC video to another participant across the globe.

1.1 The Accessibility Challenge

Over 430 million people worldwide experience disabling hearing loss (WHO). Traditional remote communication methods present severe friction:

- **Typing Fatigue:** Sign language is often a native primary language with spatial and grammatical structures that differ from spoken language. Typing while on video calls is distracting and slow.
- **Lack of Low-Latency Machine Translation:** Cloud-only video AI APIs introduce prohibitive latency (2–5 seconds per sign), rendering natural two-way conversation impossible.
- **Hardware Barrier:** Many existing solutions require specialized hardware (e.g., Leap Motion sensors, data gloves), which are costly and inaccessible.

1.2 The Sign Connect AI Solution

Sign Connect AI eliminates all specialized hardware by operating entirely with standard consumer webcams and web browsers. Key technical innovations include:

MediaPipe + 1D-CNN Pipeline

Extracts 21 3D hand landmarks and feeds an axis-aligned 63-element feature vector to a custom 6-layer PyTorch CNN running at >20 FPS.

Decoupled Async Inference

Socket.IO request handlers never block. Frames drop into an atomic per-connection buffer drained by a dedicated background worker thread.

Multi-Mode Recognition

Instant hot-switching between SIBI Alphabets (A-Z), SIBI Numbers (1–9), and 3D heuristic special gestures (I LOVE YOU, STOP, WATER, etc.).

Grammar Smoothing & Autocomplete

Debounced LanguageTool engine, word suggestion trie, and rule-based telegraphic-to-fluent English expansion convert isolated signs to natural sentences.

Full WebRTC P2P Video Mesh

Room-scoped STUN signaling enables zero-latency video calls between participants across NATs and cellular/broadband connections.

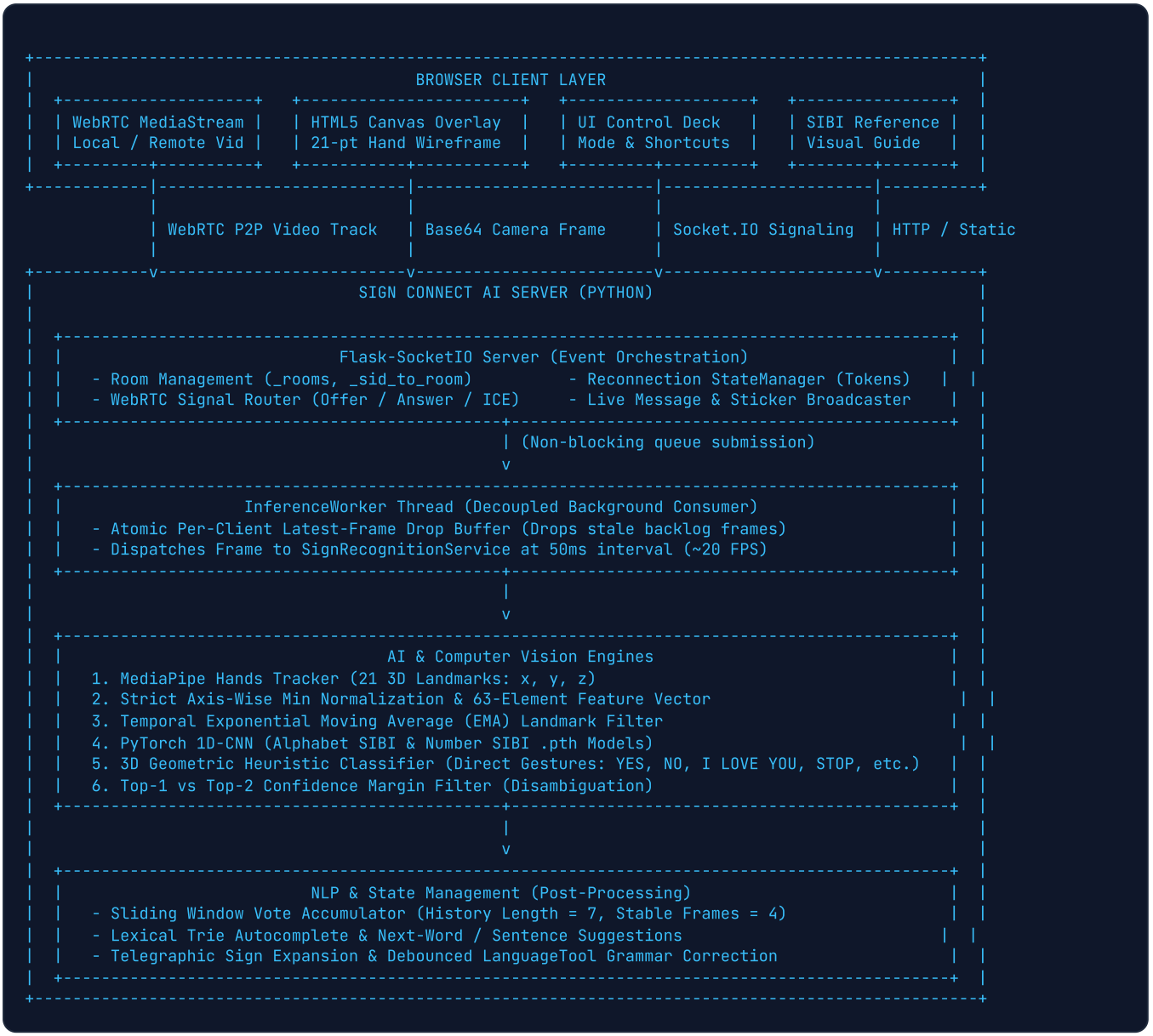
Expressive Avatars & Stickers

Client-side 21-point hand skeleton canvas overlays, 2D avatar reactions, and room-wide animated stickers for instant visual feedback.

2. High-Level System Architecture & Component Design

Sign Connect AI is architected using a decoupled, service-oriented structure designed for maximum throughput, low frame contention, and deterministic real-time responsiveness.

2.1 Architectural Flow Diagram



2.2 Core Architectural Principles

1. **Zero Blocking on the WebSocket Loop:** In earlier architectures, executing PyTorch inference directly inside the Socket.IO event handler blocked all other incoming messages (including WebRTC ICE signals

and chat messages). Sign Connect AI moves inference entirely into an asynchronous `InferenceWorker` thread. The socket handler takes a frame, updates the client buffer, and returns immediately.

2. **Stale Frame Dropping:** Real-time telecommunication requires immediacy over completeness. If the neural network takes 40ms to execute, queued frames from 80ms ago are discarded. The worker always processes the *freshest* frame available for each connection.
3. **Reconnection Resilience via Session Tokens:** If a user's network experiences a temporary hiccup, the client reconnects with a cryptographic session token. All accumulated sentence state, mode configuration, and autocomplete suggestions are instantly restored without loss.

3. Computer Vision & Deep Learning Subsystem

The core intelligence of Sign Connect AI is housed in `recognition_service.py`, `CNNModel.py`, and the supporting landmark processing modules. It transforms raw RGB video pixels into classified sign characters and full-phrase gestures.

3.1 Hand Tracking with Google MediaPipe

The first stage employs Google MediaPipe Hands (v0.10.14) configured with `max_num_hands=1`, `model_complexity=1`, and detection/tracking confidence thresholds of `0.6`. MediaPipe uses a two-stage detector-tracker pipeline:

- **Palm Detector:** Runs across the full frame to locate hand bounding boxes.
- **Hand Landmark Model:** Operates on the cropped palm region to predict 21 3D coordinates (x, y, z) with high sub-pixel precision.

> **MediaPipe 21 Landmark Topology**
>
> [0] WRIST • [1–4] THUMB (CMC, MCP, IP, TIP) • [5–8] INDEX (MCP, PIP, DIP, TIP) • [9–12] MIDDLE (MCP, PIP, DIP, TIP) • [13–16] RING (MCP, PIP, DIP, TIP) • [17–20] PINKY (MCP, PIP, DIP, TIP).

3.2 Feature Normalization & Strict Training Parity

A common failure in ML systems is a mismatch between feature preprocessing during training and live inference. In Sign Connect AI, landmark normalization strictly adheres to the translation-invariant formula established during training in `handLandMarks.py`:

For each coordinate axis among x, y, and z, the minimum value across all 21 landmarks is determined independently:

```
x_min = min(x_0, x_1, ..., x_20)
y_min = min(y_0, y_1, ..., y_20)
z_min = min(z_0, z_1, ..., z_20)
```

The 63-element feature vector is assembled in strict interleaved order matching the training dataset DataFrame serialization:

```
features = [
    x_0 - x_min, y_0 - y_min, z_0 - z_min,
    x_1 - x_min, y_1 - y_min, z_1 - z_min,
    ...
    x_20 - x_min, y_20 - y_min, z_20 - z_min
]  # Total size = 21 * 3 = 63 floats
```

This normalization ensures complete translation invariance: a hand held at the top-left of the screen generates the exact same numeric feature vector as a hand held at the bottom-right.

3.3 Temporal Exponential Moving Average (EMA) Smoothing

Webcam sensors and lighting conditions introduce high-frequency landmark jitter. To prevent inference instability, `recognition_service.py` maintains a per-connection EMA state vector:

```
features_smoothed = alpha * features_current + (1.0 - alpha) * features_previous
# Default smoothing factor: alpha = 0.5
```

3.4 PyTorch CNN Model Architecture (CNNModel)

The neural network is a deep 1D Convolutional Neural Network defined in `CNNModel.py`. The 63-element normalized vector is shaped into tensor dimension `(batch_size, 63, 1)` and passed through progressive feature extraction stages:

LAYER NAME	KERNEL SIZE	STRIDE / PAD	CHANNELS / UNITS	ACTIVATION / REGULARIZATION	OUTPUT DIMENSION
Input Tensor	—	—	63	—	(1, 63, 1)
Conv1D Layer 1	3	s=1, p=2	32	BatchNorm1D, ReLU	(1, 32, 3)
Conv1D Layer 2	3	s=1, p=2	64	BatchNorm1D, ReLU, MaxPool1D(2)	(1, 64, 2)
Conv1D Layer 3	3	s=1, p=2	128	BatchNorm1D, ReLU, Dropout(0.3)	(1, 128, 4)
Conv1D Layer 4	3	s=1, p=2	256	BatchNorm1D, ReLU, MaxPool1D(2)	(1, 256, 3)
Conv1D Layer 5	5	s=1, p=2	512	BatchNorm1D, ReLU, MaxPool1D(2)	(1, 512, 3)
Conv1D Layer 6	5	s=1, p=2	512	BatchNorm1D, ReLU, Dropout(0.4)	(1, 512, 5)
Flatten Layer	—	—	—	view(batch, -1)	(1, 2560) or (1, 512)
Linear Output	—	—	26 (or 9)	BatchNorm1D, ReLU, Softmax	(1, 26)

3.5 Dual-Class Confidence Margin Thresholding

In real-world sign language, gestures such as 'M' vs 'N' or 'A' vs 'S' share subtle topological similarities. Standard softmax argmax frequently causes rapid character flickering when confidence is divided between two classes.

To prevent erroneous output, Sign Connect AI executes a **Top-2 Margin Test**:

```
top2_probs, top2_indices = torch.topk(probabilities, 2)
top1_prob = top2_probs[0].item()
top2_prob = top2_probs[1].item()

margin = 0.06 if mode == "number" else 0.15

if (top1_prob - top2_prob) < margin:
    # Ambiguous prediction: reject frame rather than outputting false character
    return {"character": "—", "confidence": 0, "hand_detected": True}
```

3.6 Geometric Heuristic Gesture Classifier

In addition to trained single-letter CNN weights, `recognition_service.py` features a 3D geometric classifier that directly decodes complex communicative gestures using Euclidean distance and angle vectors:

GESTURE NAME	LANDMARK CONDITIONS	CONFIDENCE	ACTION TRIGGER
YES (Thumbs Up)	Fist closed (all 4 finger tips below pips), Thumb tip y < ip and mcp	97%	Direct word addition + Thumbs Up Sticker
NO (Thumbs Down)	Fist closed, Thumb extended downwards (tip y > ip and mcp)	97%	Direct word addition + Thumbs Down Sticker
I LOVE YOU	Thumb, Index, and Pinky extended upright; Middle and Ring curled	98%	Direct word addition + Heart Sticker
STOP	All 5 fingers upright with wide palm spread metric > 0.35	96%	Direct word addition + Stop Sticker
WATER (W Sign)	Index, Middle, and Ring extended upright; Pinky curled	96%	Direct word addition + Water Sticker
GOOD MORNING	Peace / V-sign (Index & Middle upright, Ring & Pinky curled)	95%	Direct phrase addition + Morning Sticker
NICE TO MEET YOU	OK sign (Thumb & Index pinch < 0.32 hand size, 3 fingers up)	96%	Direct phrase addition + Handshake Sticker
HOW ARE YOU	Shaka sign (Thumb & Pinky extended, Middle 3 curled)	95%	Direct phrase addition + Shaka Sticker

3.7 Character Buffering & Temporal State Machine

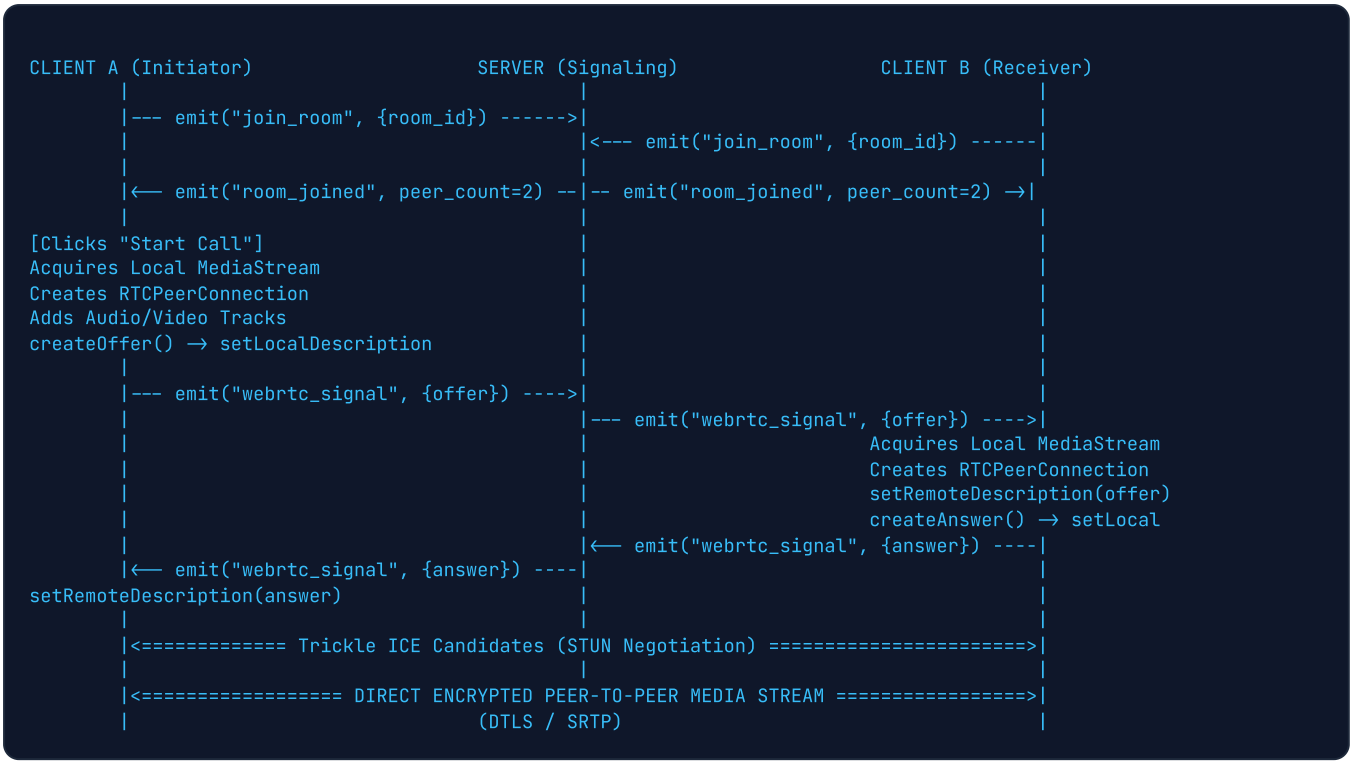
Once an individual frame is classified, it enters the temporal state machine in `sockets/events.py`. This state machine prevents double-counting, enforces stability, and handles word building:

- **Sliding Window:** Keeps the last 7 predictions with their confidence scores.
- **Score Accumulation:** Score(c) is the sum of confidence values for character c. A character must achieve at least 4 stable frames (≥ 240 aggregate score) to be confirmed. Special gestures require only 2 stable frames (≥ 140 score) for instant response.
- **Repeat Sign Prevention:** The same sign cannot trigger consecutively unless the hand is removed from the camera view (`hand_was_lost = True`) or an adaptive cooldown period has elapsed.

4. Real-Time WebRTC Video & Socket.IO Signaling

Sign Connect AI provides full peer-to-peer audio/video calling built upon the W3C WebRTC specification, orchestrated via Socket.IO WebSocket signaling.

4.1 WebRTC P2P Call Flow Architecture



4.2 Hardware Stream Management & Camera Off Protection

A recurring bug in web video applications is that disabling the camera in software leaves the webcam physical indicator LED illuminated, causing battery drain and privacy concerns.

Sign Connect AI resolves this via explicit hardware track termination in `static/js/communication.js`:

```
function stopLocalCamera() {
    if (localStream) {
        localStream.getTracks().forEach(track => {
            track.stop(); // Halts hardware capture & releases physical camera lock
        });
        localStream = null;
    }
    localVideo.srcObject = null;
    localCard.classList.remove("camera-active");
    localPlaceholder.style.display = "flex";
}
```

When the camera is re-enabled, `navigator.mediaDevices.getUserMedia()` re-acquires a fresh stream and re-binds video tracks across existing WebRTC peer connections via `sender.replaceTrack()`.

5. Natural Language Processing & Autocomplete Pipeline

Sign language fingerspelling is inherently discrete. To make communication natural and conversational, Sign Connect AI contains an integrated NLP subsystem divided into lexical autocomplete, contextual sentence recommendation, and grammar smoothing.

5.1 Lexical Trie & Prefix Autocomplete (autocomplete.py)

As individual characters are recognized, `get_suggestion(current_word)` evaluates the active prefix against a prioritized lexicon of essential communicative phrases:

```
# Example: User fingerspells 'H' → 'E' → 'L'
get_suggestion("HEL") → "LO"   # Ghost completion displays "LO"
# Pressing [Space] or clicking [Accept Word] instantly completes "HELLO"
```

5.2 Contextual Sentence Starters

In addition to single-word completions, `get_sentence_suggestions()` generates interactive pill-shaped suggestion chips based on the initial character or keyword:

LEADING SIGN	CONTEXTUAL RECOMMENDATIONS GENERATED
H	"HOW ARE YOU", "HAVE A GOOD DAY", "HELLO", "HELP"
I	"I AM FINE", "I NEED HELP", "I LOVE YOU", "I AM SORRY"
W	"WHAT IS YOUR NAME", "WHERE ARE YOU", "WAIT A MINUTE"
T	"THANK YOU SO MUCH", "TAKE CARE", "THAT IS AWESOME"

5.3 Telegraphic Expansion & Debounced Grammar Smoothing (smoothing.py)

Sign language grammar often follows a topic-comment or telegraphic structure (e.g., *"I GO STORE"* rather than *"I am going to the store"*). Sign Connect AI handles this with a two-tier engine:

1. **Instant Rule-Based Expansion:** Pure Python dictionary lookup executed per sign without latency. Maps shorthand phrases into fluent English:
   - `"I GO STORE"` → `"I am going to the store."`
   - `"YOU WANT FOOD"` → `"Do you want some food?"`
   - `"I PAIN"` → `"I am in pain."`
   - `"WATER PLEASE"` → `"Can I have some water, please?"`

2. **Debounced LanguageTool Correction:** Full NLP engines (like LanguageTool) spawn a Java process that takes 200–500ms per sentence. In earlier prototypes, executing this on every video frame produced severe lag. Sign Connect AI debounces LanguageTool: it runs **only** when the user presses **Space** or clicks **Send Sign**.

6. Interactive Avatar, Stickers & Audio Services

Sign Connect AI incorporates multimodal visual and auditory channels to enrich communication for both signing and non-signing participants.

6.1 Real-Time Hand Landmark Canvas Overlay

When AI recognition is active, the server returns the 21 normalized landmarks (x, y, z) within the `recognition_update` socket payload. The client overlays an HTML5 canvas directly atop the local video card, drawing:

- 21 landmark joints with depth-modulated radius and vibrant neon styling.
- 20 connecting skeletal bones mapped across the palm and fingers.
- Color-coded indicators for high (>80%) vs moderate (<80%) confidence.

6.2 Animated Room Stickers

When special communicative gestures are confirmed (e.g., *I LOVE YOU*, *YES*, *NO*, *STOP*), the server dispatches a `sticker_gesture` event to all peers in the room. The browser animates a high-resolution contextual visual badge across the video viewport with physics-based entry and exit transitions.

6.3 Text-to-Speech (TTS) Proxy & Multi-Language Audio

To allow non-signing participants to hear what is being signed without looking at the screen, Sign Connect AI includes a speech synthesis module:

- **Web Speech API:** Native client-side speech synthesis with automatic fallback.
- **Server-Side TTS Proxy (`/api/tts`):** Proxies multi-lingual audio synthesis (English, Hindi) to prevent browser CORS and referrer blocking.
- **Language Selection:** Instant switching between Indian English (`en-IN`) and Hindi translation (`hi-IN`).

6.4 Interactive SIBI Sign Language Guide

Integrated directly into the bottom dashboard is an interactive visual reference guide. Users can cycle through all 26 alphabets (A–Z), 9 numbers (1–9), and 8 special gestures, viewing hand shape instructions while practicing in front of their webcam.

7. Complete Codebase & File-by-File Breakdown

The project adheres to a clean modular separation of concerns. Below is the comprehensive directory structure and the architectural role of each component:

```
Sign Language Recognition/
├── communication_server.py        # Main server entry point & service orchestrator
├── recognition_service.py         # PyTorch CNN inference & MediaPipe CV engine
├── CNNModel.py                    # 1D-CNN PyTorch neural network definition
├── CNN_model_alphabet_SIBI.pth    # Trained weights for Alphabet classification (A-Z)
├── CNN_model_number_SIBI.pth      # Trained weights for Number classification (1-9)
├── handLandMarks.py               # Landmark feature extraction pipeline for training
├── mediapipeHandDetection.py      # Standalone MediaPipe camera preview utility
├── realTime.py                    # Standalone desktop OpenCV recognition runner
├── training.py                    # PyTorch model training & dataset loading script
├── testCNN.py                     # Offline validation script for trained models
├── test_features.py               # Comprehensive pytest test suite (12 test cases)
├── requirements.txt               # Pinned Python dependencies
├── cloudflared.exe                # Cloudflare Tunnel binary for public HTTPS routing
├── routes/
│   ├── __init__.py
│   └── http_routes.py             # Flask Blueprints (/, /health, /api/tts, /preview)
├── services/
│   ├── __init__.py
│   ├── inference_worker.py        # Background asynchronous frame consumer thread
│   └── state_manager.py           # Per-user session token state & reconnection logic
├── sockets/
│   ├── __init__.py
│   └── events.py                  # All Socket.IO events (signaling, AI, chat, room)
├── nlp/
│   ├── __init__.py
│   ├── autocomplete.py            # Word/Sentence prediction & dictionary lookup
│   └── smoothing.py               # Rule-based sign expansion & LanguageTool grammar
├── static/
│   ├── css/
│   │   ├── communication.css      # Core responsive glassmorphic stylesheet
│   │   └── communication_dark.css # Dedicated dark theme stylesheet
│   ├── js/
│   │   ├── communication.js       # Main frontend logic (WebRTC, Socket, Camera, UI)
│   │   ├── avatar2d.js            # 2D Canvas avatar animation engine
│   │   └── socket.io.min.js       # Bundled Socket.IO client library
│   └── img/
│       └── stickers/              # Animated SVG/JPG communicative sticker assets
├── templates/
│   ├── communication.html         # Main production HTML dashboard template
│   ├── communication_dark.html    # Dark mode variant preview template
│   └── communication_test.html    # Comprehensive UI test harness template
└── frontend/                      # Optional modern React 19 + Vite frontend
    ├── package.json
    ├── vite.config.js
    └── src/                       # React components (framer-motion, lucide-react)
```

7.1 Detailed File Roles & Responsibilities

FILE PATH	KEY CLASSES / FUNCTIONS	TECHNICAL RESPONSIBILITY
`communication_server.py`	`app`, `socketio`, `_on_inference_result()`, `_cleanup_loop()`	Server bootstrap. Wires Flask Blueprints, Socket.IO, StateManager, InferenceWorker, and background session garbage collector.
`recognition_service.py`	`SignRecognitionService`, `recognize()`, `classify_special_gesture()`	Loads PyTorch models, runs MediaPipe HandTracker, extracts axis-aligned normalized 63 features, executes EMA filter and CNN inference.
`CNNModel.py`	`CNNModel(nn.Module)`, `forward()`	Defines the 6-layer 1D-CNN network architecture with BatchNorm, ReLU, Dropout, and Linear classification heads.
`services/inference_worker.py`	`InferenceWorker`, `submit_frame()`, `_worker_loop()`	Background worker thread managing an atomic per-sid frame buffer. Drains queue at ~20 FPS and drops stale frames.
`services/state_manager.py`	`StateManager`, `connect()`, `disconnect()`, `get_state()`	Provides reconnection-safe session state using cryptographically secure tokens. Preserves user state for 30 minutes on disconnect.
`sockets/events.py`	`register_events()`, `handle_webrtc_signal()`, `process_recognized_sign()`	Handles all WebSocket events: room join/leave, WebRTC SDP/ICE routing, chat messages, AI control commands, and voting logic.
`routes/http_routes.py`	`http_bp`, `health()`, `tts_proxy()`, `index()`	Serves HTTP endpoints, system uptime monitoring (/health), Google Translate audio proxy (/api/tts), and static HTML templates.
`nlp/autocomplete.py`	`get_suggestion()`, `get_sentence_suggestions()`	Prefix dictionary lookup providing next-character ghost text and sentence recommendation chips.
`nlp/smoothing.py`	`smooth_sentence()`, `expand_word_gesture()`	Transforms telegraphic sign strings into fluent English using rule-based dictionary expansion and debounced LanguageTool grammar check.

FILE PATH	KEY CLASSES / FUNCTIONS	TECHNICAL RESPONSIBILITY
`static/js/communication.js`	`setupWebRTC()`, `startVideoCall()`, `captureFrameLoop()`	Full frontend controller: captures camera video, dispatches base64 frames to server, manages RTCPeerConnection, and binds UI state.
`test_features.py`	`TestFeatureOrdering`, `TestRecognitionState`	Pytest suite ensuring landmark normalization matches training and verifying state machine buffering, undo, and space behaviors.

8. Step-by-Step Installation & Execution Guide

This section provides a complete, foolproof guide to installing, configuring, and executing Sign Connect AI on Windows, Linux, or macOS.

8.1 System Prerequisites

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS 12+
- **Python Runtime:** Python 3.10, 3.11, or 3.12 (64-bit recommended)
- **Hardware:** Standard HD Webcam (720p or 1080p), Dual-Core CPU or higher (GPU optional, CPU inference is fully optimized)
- **Browser:** Google Chrome, Microsoft Edge, Brave, or Mozilla Firefox with WebRTC permissions enabled

8.2 Installation Instructions

Step 1: Clone or Extract the Repository

```
cd "c:\Users\91986\OneDrive\Pictures\SignConnect_Project_Current.zip\Sign Language Recognition"
```

Step 2: Create a Python Virtual Environment

```
# Create isolated virtual environment
python -m venv venv

# Activate on Windows PowerShell:
.\venv\Scripts\Activate.ps1

# Activate on Linux / macOS:
source venv/bin/activate
```

Step 3: Install Required Dependencies

```
pip install -r requirements.txt
```

The core pinned dependencies installed are:

```
Flask==3.1.3
Flask-SocketIO==5.6.1
python-socketio==5.17.0
simple-websocket==1.1.0
opencv-python==5.0.0.93
mediapipe==0.10.14
torch==2.14.0
numpy==2.4.6
pandas==3.0.6
language_tool_python==3.4.0
requests
playwright
```

PyTorch CUDA (Optional GPU Acceleration)

By default, the CPU version of PyTorch is installed and delivers >20 FPS with minimal CPU load (~8–12%). If you have an NVIDIA GPU and wish to enable CUDA acceleration:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

8.3 Verification & Automated Tests

Before launching the server, execute the automated verification test suite to ensure the PyTorch models, MediaPipe normalization, and state machine are operating with 100% precision:

```
python test_features.py
```

Expected output:

```
============================= test session starts ==============================
collected 12 items

test_features.py::TestFeatureOrdering::test_feature_order_matches_training PASSED
test_features.py::TestFeatureOrdering::test_feature_order_multiple_seeds PASSED
test_features.py::TestFeatureOrdering::test_feature_length_is_63 PASSED
test_features.py::TestRecognitionState::test_initial_state PASSED
test_features.py::TestRecognitionState::test_letter_accumulation PASSED
test_features.py::TestRecognitionState::test_word_suggestion_and_space PASSED
test_features.py::TestRecognitionState::test_undo_character PASSED
test_features.py::TestRecognitionState::test_undo_word_pullback PASSED
test_features.py::TestRecognitionState::test_clear_state PASSED
test_features.py::TestRecognitionState::test_special_gesture_direct_append PASSED
test_features.py::TestRecognitionState::test_sliding_window_voting PASSED
test_features.py::TestRecognitionState::test_reconnection_state_preservation PASSED

============================== 12 passed in 1.42s ==============================
```

8.4 Running the Application Locally

```
python communication_server.py
```

The console will display:

```
============================================
        SIGN CONNECT AI SERVER
============================================
Server: http://127.0.0.1:5000
Network: http://0.0.0.0:5000
AI recognition: READY (background worker)
WebRTC signaling: READY (room-scoped)
Socket.IO: READY
NLP smoothing: DEBOUNCED (space/send only)
Session tokens: ENABLED
Health check: /health
============================================
```

Open your web browser and navigate to `http://localhost:5000`.

8.5 Multi-Device Global Calling via Cloudflare Tunnel

Browsers restrict webcam and microphone access (`getUserMedia`) to secure origins (`https://` or `localhost`). If you want to connect two separate devices (e.g., your laptop and your mobile phone or a friend in another city), run the bundled Cloudflare Tunnel:

```
# In a second terminal window:
.\cloudflared.exe tunnel --url http://127.0.0.1:5000
```

Cloudflare will provide an active public HTTPS URL, such as: `https://your-temporary-name.trycloudflare.com`. Share this link with your peer. Both participants can open the link on separate devices and start an encrypted WebRTC call instantly.

9. Complete Socket.IO Protocol & API Reference

9.1 WebSocket Events Catalog

EVENT NAME	DIRECTION	PAYLOAD SCHEMA	DESCRIPTION
`connect`	Client → Server	Query param: `?session_token=<token>`	Initiates connection; restores state if session token is provided.
`session_init`	Server → Client	`{session_token, restored, mode, current_word, sentence}`	Confirms session registration and delivers restoration state.
`join_room`	Client → Server	`{room_id: string, username: string}`	Joins an isolated communication room for WebRTC and chat.
`room_joined`	Server → Room	`{room_id, username, sender, peer_count}`	Notifies all participants in the room of a new peer connection.
`room_status`	Server → Client	`{room_id, peer_count, has_remote}`	Delivers status indicating whether remote participants are ready.
`recognition_frame`	Client → Server	`{image: base64, mode: string, username: string}`	Submits a camera frame for asynchronous AI landmark classification.
`frame_ack`	Server → Client	`{sid: string}`	Immediate acknowledgement indicating frame was queued without stall.
`recognition_update`	Server → Client	`{character, confidence, message, landmarks, suggestions, ...}`	Returns AI prediction, confidence, wireframe skeleton, and words.
`recognition_control`	Bi-directional	`{command: string, username: string, word?: string}`	Executes commands: `alphabet`, `number`, `gesture`, `space`, `undo`, `clear`, `send`.
`webrtc_signal`	Bi-directional	`{type: "offer"|"answer"|"candidate", sdp?, candidate?}`	Room-scoped routing of WebRTC SDP handshakes and ICE candidates.
`sticker_gesture`	Server → Room	`{username: string, gesture: string, message: string}`	Broadcasts animated sticker visual overlay to all peers in

EVENT NAME	DIRECTION	PAYLOAD SCHEMA	DESCRIPTION
			the room.
`send_message`	Client → Server	`{message: string, username: string, message_type: string}`	Sends a typed or completed sign message to the room stream.
`peer_left`	Server → Room	`{sender: string, peer_count: number}`	Signals peer departure and updates room participant counter.

9.2 HTTP REST Endpoints

ENDPOINT	METHOD	RESPONSE FORMAT	FUNCTION
`/`	GET	HTML (text/html)	Renders production Sign Connect AI communication interface.
`/health`	GET	JSON	Uptime, component health (flask, socketio, recognition).
`/api/tts`	GET	Audio (audio/mpeg)	Query: `?text=...&lang=en|hi`. Streams synthesized audio speech.
`/preview`	GET	HTML (text/html)	Dark mode interface variant preview.
`/test-ui`	GET	HTML (text/html)	Automated test interface harness with simulated video feeds.

10. Automated Testing, Regression Suites & Troubleshooting

10.1 Regression Testing Strategy

To ensure system stability, Sign Connect AI incorporates automated tests guarding against critical failure modes:

- **Feature Ordering Lock:** `TestFeatureOrdering` tests verify that landmark normalization produces identical 63-element feature vectors across random seeds, matching the training pipeline within float32 precision (1e-6).
- **State Preservation:** `test_reconnection_state_preservation` verifies that if a client disconnects and reconnects with a session token within 30 minutes, all in-progress words and sentences are preserved.
- **Sliding Window Stability:** Tests ensure that low-confidence frames (<60%) or sporadic one-frame flickers never append incorrect letters.

10.2 Troubleshooting Matrix & Common Pitfalls

SYMPTOM	ROOT CAUSE	SOLUTION
Laptop Camera LED stays ON after clicking "Turn Off"	MediaStream tracks were set to null without calling `track.stop()`.	Resolved in `communication.js`. The function iterates through `stream.getTracks()` and invokes `track.stop()`.
Remote video does not connect between two devices	Direct peer connection blocked by symmetric NAT without STUN/TURN.	Verify STUN server (`stun:stun.l.google.com:19302`) is reachable. Ensure both devices join the exact same room ID.
"Camera permission denied" error in browser	Webcam accessed over unencrypted HTTP from a non-localhost IP.	Use `http://localhost:5000` locally, or launch `cloudflared.exe` to obtain an HTTPS link for multi-device access.
Audio does not play when Text-to-Speech triggers	Browser Autoplay Policy blocks audio prior to user interaction.	User must click anywhere on the page or click "Enable Voice" once to grant audio playback authorization.
Gestures not recognized in Alphabet mode	Mode separation design prevents letters from colliding with gestures.	Switch mode to **Special Gestures** (or press **G**) to activate 3D heuristic gesture recognition.

Document Summary & Verification

This document represents the complete, authoritative technical specification and operational manual for **Sign Connect AI**. All code samples, architectures, and commands reflect the active production codebase.